Les types & les structures

- Le mot-clé **typedef**
- Les énumérations
- Les structures
- La composition
- Les tableaux de structure
- Les structures en paramètres de fonctions

Rado Rakotonarivo (Cours d'Asma Gabis)
rado.rakotonarivo@efrei.fr

P1 – 2024/2025

01

Création d’un nouveau type en langage C

01

Création d'un nouveau type en langage C

Pourquoi ?

Création de nos propres types : à partir des types déjà intégrés dans le langage C ou une liste de valeurs.

Généricité : Utiliser des programmes identiques opérant sur des données différentes.

Création d'un nouveau type en langage C

Comment ?

En utilisant le mot clé : **typedef**

Syntaxe

typedef type_existant t_nouveau_type ;

On conviendra de toujours préfixer le nom du nouveau type par 't_'

Création d'un nouveau type en langage C

Exemples

Exemple 1

```c
typedef int t_entier;
t_entier x = 3;
printf("%d", x);
```

Exemple 2

```c
typedef char* t_string;
t_string cours = "programmation";
printf("%s", cours);
```

Exemple 3

```c
#include <stdlib.h>

typedef  double  t_items_type;

t_items_type * tab = (t_items_type *) malloc (10 * sizeof(t_items_type));
```

Allocation de 10 cases de type double

Changer simplement double par int

Allocation de 10 cases de type int

```c
#include <stdlib.h>

typedef  int t_items_type;

t_items_type * tab = (t_items_type *) malloc (10 * sizeof(ITEMS_TYPE));
```

Création d'un nouveau type en langage C

Où sont définis les nouveaux types ?

Tous les nouveaux types créés sont définis dans le fichier .h du module concerné.

Hormis la généricité, **typedef** devient vraiment utile lorsqu'il est associé à deux choses:

- les types énumérés
- les structures.

02

Les types énumérés

02

Les types énumérés

Définition

- Une énumération est une liste de valeurs possibles que peut prendre une variable
- Pour créer une énumération en langage C, le mot-clé **enum** est utilisé
- À partir d'une énumération, il est possible de créer un nouveau type en C grâce au **typedef**
- On conviendra que les types **enum** seront préfixé par 'e_'

Exemple 1

```
#include <stdio.h>

typedef enum
{
    lundi, mardi, mercredi, jeudi, vendredi, samedi, dimanche
} e_semaine;

int main()
{
    e_semaine jour = lundi;


    if (jour == lundi)
        printf("C'est lundi !");
    else
        printf("Ce n'est pas lundi !");
    return 0;
}
```

Les types énumérés

Manipuler les valeurs d'une énumération

- Chaque élément de l'énumération est associé à un entier, en interne (implicitement).

- 0 est donc associé au premier élément, 1 est associé au deuxième élément, …etc.

Exemple 2

```
#include <stdio.h>

typedef enum
{
    rouge, vert, bleu, jaune, noir, blanc
} e_couleur;

int main()
{
    e_couleur c ;
    printf("Saisir une couleur : ");
    scanf("%d", &c);
    if (c == 2)
        printf("Bleu !");
    else
        printf("Pas bleu !");
    return 0;
}
```

Les types énumérés

Associer une valeur précise

Il est aussi possible d'associer une valeur précise (choisie par le développeur) à chaque élément de l'énumération

Exemple 3

```
#include <stdio.h>

typedef enum
{
    faible = 10, moyen = 50, fort = 100
} e_volume;

int main()
{
    e_volume v = faible ;

    if (v == 100)
        printf("Le volume est fort !");
    else
        printf("Le volume n'est pas fort !");
    return 0;
}
```

03

Les structures

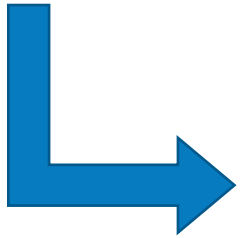
03

Les structures

Besoin

Nous avons souvent besoin de regrouper, **dans une même entité**, différentes propriétés et/ou informations d'un même "objet".

- Un étudiant : nom, prénom, numéro de carte d'étudiant, adresse , notes
- Un joueur vidéo : pseudo, score, nombre de vies
- Une date : jour, mois , année
- Un article (dans un magasin) : code, libellé, prix unitaire, quantité en stock



- Est-il possible d'utiliser un tableau pour regrouper les différentes propriétés ?
- Comment faire pour représenter plusieurs objets ? Avec plusieurs tableaux ?!!!!

Les structures

Définition

- Une structure est une **agrégation** d'un ensemble d'informations appelées membres/attributs/champs de types différents.

- Une structure permet de définir un type composé choisi par le développeur.

- La définition d'une structure nécessite de préciser son nom et les membres qu'elle contient.

Les structures

Syntaxe

- Pour définir une structure en langage C, le mot clé **struct** est utilisé

```
struct nom_structure
{
    type_att1 nom_att1;
    type_att2 nom_att2;
    .....
} ;
```

Un nouveau type est défini :

- Son nom est : **struct nom_strcuture**
- Il peut être utilisé comme n'importe quel autre type primitif du langage C :
  - Pour déclarer des variables
  - Dans les paramètres des fonctions
  - Pour les valeurs d'un tableau
  - Pour un pointeur

Les structures

Exemples

```c
// Définition d'un nouveau type : struct date

struct date
{
    int jour;
    int mois;
    int annee;
};
```

```c
// Le type struct date est utilisé pour un paramètre de type pointeur

void lireDate(struct date *d)
{

    ...

}
```

```c
// Utilisation du type struct date comme type des paramètres d'une fonction

int compareDates(struct date d1, struct date d2)
{

    ...

}
```

```c
int main()
{
    // Déclaration de variables de types struct date
    struct date d1, d2;

    // Déclaration d'un tableau de type struct date
    struct date TD[50];
    ...
    return 0;
}
```

Les structures

Utiliser un type structure sans struct

Pour éviter d'utiliser le mot clé **struct** à chaque déclaration d'une variable du nouveau type créé, nous disposons de deux options.

Option 1

```
struct nom_structure
{
        type_att1 nom_att1;
        type_att2 nom_att2;
        .....
} ;
typedef struct nom_structure t_nouveau_type;
```

Option 2

```
typedef struct
{
        type_att1 nom_att1;
        type_att2 nom_att2;
        .....
} t_nouveau_type;
```

Les structures

Utiliser un type structure sans struct

Exemple

Avec Option 1

```c
struct date
{
    int jour;
    int mois;
    int annee;
};
typedef struct date t_date;
int compareDates(t_date d1, t_date d2){
    ...
}
void lireDate(t_date *d){
    ...
}
int main(){
    t_date d1, d2;
    t_date TD[50];
    ...
    return 0;
}
```

Avec Option 2

```c
typedef struct
{
    int jour;
    int mois;
    int annee;
} t_date;
int compareDates(t_date d1, t_date d2){
    ...
}
void lireDate(t_date *d){
    ...
}
int main()
{
     t_date d1, d2;
     t_date TD[50];
    ...
    return 0;
}
```

Les structures

Représentation en mémoire

- Lorsqu'une variable est déclarée de type structure, elle est stockée comme étant un bloc d'attributs.

```
typedef struct
{
    type_att1 nom_att1;
    type_att2 nom_att2;
    .....
} t_nouveau_type;
t_nouveau_type nom_variable;
```

En mémoire

<table>
<tr><td rowspan="3">nom_variable</td><td>nom_att1</td></tr>
<tr><td>nom_att2</td></tr>
<tr><td>....</td></tr>
</table>

Les structures

Initialiser les attributs d'une structure

Comme pour un tableau, il est possible d'initialiser les attributs (membres) d'une structure en utilisant les **{}**

```
typedef struct
{
    char nom[255];
    int age;
} t_personne;

int main()
{
    // Initialisation la personne p1 a pour nom Julia, et pa
    t_personne p1 = {"Julia", 19};

    // Initialisation de p2 avec un nom vide et un age à 0
    t_personne p2 = {"", 0};
    return 0;
}
```

Les attributs doivent être initialisés **dans l'ordre** dans lequel ils figurent dans la structure.

Les structures

Accéder aux attributs d'une structure

- L'accès aux attributs d'une variable de type structure se fait par l'opérateur le point (« . »)

- Saisir (remplir) une structure ➔ Saisir les attributs un par un (comme pour un tableau)

- Afficher une structure ➔ Afficher les attributs un par un (comme pour un tableau).

```c
#include <stdio.h>

typedef struct
{
    int jour;
    int mois;
    int annee;
} t_date;

int main()
{
    // Déclaration d'une variable de type DATE
    t_date d;

    // Saisir une date sous la forme jour/mois/annee
    printf("Saisir la date jour/mois/annee : ");
    scanf("%d/%d/%d", &d.jour, &d.mois, &d.annee);

    // Afficher la date sous la forme jour/mois/annee
    printf("La date du jour est : %d/%d/%d \n", d.jour, d.mois, d.annee);
    return 0;
}
```

Pointeur sur une structure

- Un pointeur sur un type structure se crée de la même manière qu'un pointeur sur un type intégré dans le langage C (int, double, …)

t_nom_structure * pointeur = NULL ;

- L'accès à un attribut d'une structure à partir du pointeur se fait de deux manières :
  - En utilisant le point : **(*pointeur).nom_attribut**
  - En utilisant la flèche (->) pour une syntaxe plus simple : **pointeur -> nom_attribut**

Notez qu'en utilisant ->, on ne met pas l'* à côté du nom du pointeur

Les structures

Pointeur sur une structure

Exemple

```c
#include <stdio.h>

typedef struct
{
    int jour;
    int mois;
    int annee;
} t_date;

int main()
{
    // Déclaration d'une variable de type DATE
    t_date d1;
    // Déclaration d'un pointeur sur DATE
    t_date *d = &d1;

    // Saisir le jour et le mois
    printf("Saisir le jour : ");
    scanf("%d", &(*d).jour); // Accès à l'attribut jour par le point
    printf("Saisir le mois : ");
    scanf("%d", &(d->jour)); // Accès à l'attribut jour par la flèche

    // Afficher le jour
    printf("jour : %d \n", (*d).jour); // Accès à l'attribut jour par le point
    // Afficher le mois
    printf("mois : %d \n", d->jour); // Accès à l'attribut jour par la flèche
    return 0;
}
```

Les structures

Accès à un attribut de type pointeur

- L'accès à un attribut de type pointeur se fait par le (.) si la variable est de type structure

- L'accès à un attribut de type pointeur se fait par la (->) si la variable est de type pointeur sur une structure

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
typedef struct
{
    char* nom;
    int age;
} t_personne;

int main()
{
    t_personne pers;
    t_personne *pointeur_pers = &pers;
    printf("Affectation des informations en utilisant une variable de type structure \n");
    pers.nom = malloc(40 * sizeof(char));
    strcpy(pers.nom, "Albert");
    pers.age = 33;
    printf("Nom : %s \nAge : %d \n", pers.nom, pers.age);
    printf("Modification des informations en utilisant un pointeur sur une structure \n");
    strcpy(pointeur_pers->nom, "Jean");
    pointeur_pers->age = 23;
    printf("Nom : %s \nAge : %d \n", pointeur_pers->nom, pointeur_pers->age);
    return 0;
}
```

04

La composition

04

La composition

Définition

Une structure peut avoir un membre qui est lui même de type structure

Exemple

```
typedef struct
{
    int jour;
    int mois;
    int annee;
} t_date;
```

```
typedef struct
{
    char prenom[20];
    char nom[20];
    t_date dNaiss;
} t_etudiant
```

Ici l'attribut **dNaiss** est composé de :
- Jour
- Mois
- annee

La composition

Accès à un attribut d'un attribut

Pour accéder à un attribut d'un attribut d'une variable de type structure :

- Commencer l'accès à partir **de la variable principale de type structure**
- **Par cascade** , accéder aux attributs en respectant les règles énoncées plus haut :
  - Si l'attribut est une simple variable, utiliser l'accès par (.)
  - Si l'attribut est un pointeur , utiliser l'accès par (->)

```c
#include <stdio.h>

typedef struct{
    int jour;
    int mois;
    int annee;
} t_date;
typedef struct{
    char prenom[20];
    char nom[20];
    DATE dNaiss;
} t_etudiant;

int main(){
    t_etudiant st ;
    t_etudiant *pointeur_st = &st;
    printf("Saisir le jour de naissance ");
    scanf("%d", &st.dNaiss.jour);
    printf("Saisir le mois de naissance ");
    scanf("%d", &pointeur_st->dNaiss.mois);

    printf("Jour de naissance : %d \nMois de naissance : %d \n",
st.dNaiss.jour, pointeur_st->dNaiss.mois);
    return 0;
}
```

05

Les tableaux de structures

05

Les tableaux de structure

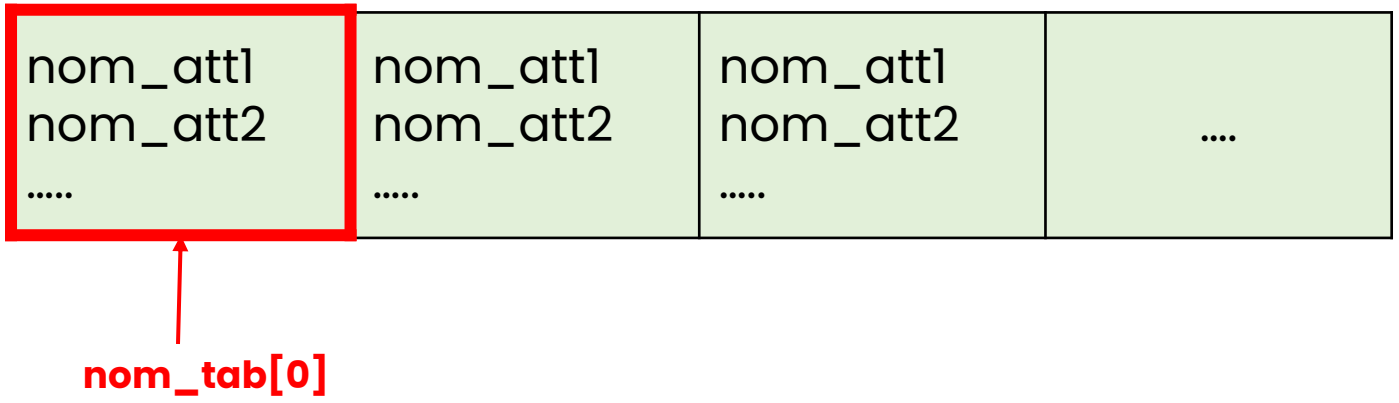
Définition

Comme la définition d'une nouvelle structure est une définition d'un nouveau type, il est donc possible de déclarer des tableaux (statiques ou dynamiques) de types structures.

Syntaxe

```
typedef struct
{
        type_att1 nom_att1;
        type_att2 nom_att2;
        .....
} t_nouveau_type;
t_nouveau_type nom_tab[taille];
```

En mémoire

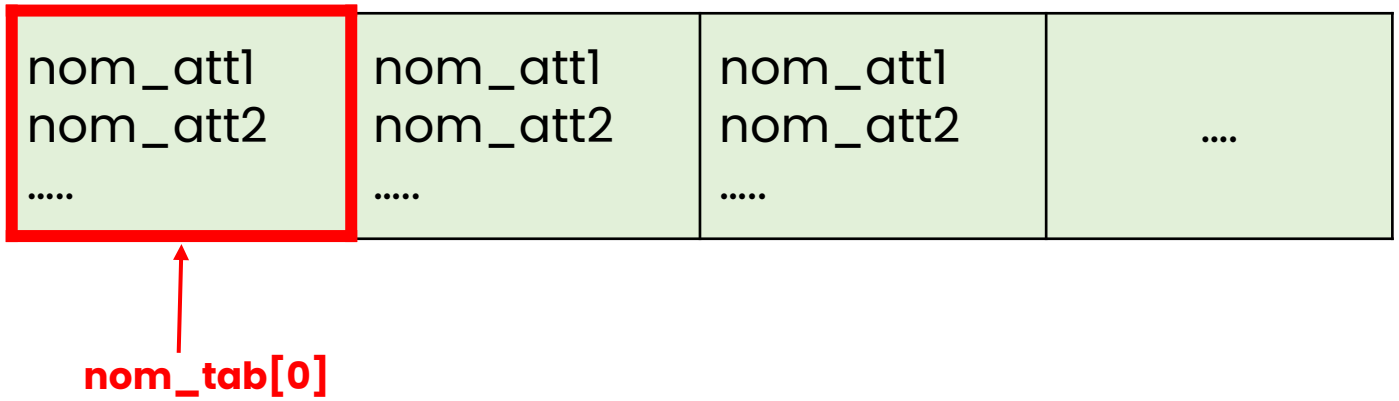


Les tableaux de structure

Accès à un attribut d'une structure

Pour accéder à un attribut d'une structure dans un tableau :

- Utiliser l'indice pour accéder à la case souhaitée dans le tableau
- À partir de la case, utiliser les règles énoncées précédemment pour accéder aux attributs



Les tableaux de structure

Exemple

```c
#include <stdio.h>
#include <stdlib.h>


typedef struct
{
    int jour;
    int mois;
    int annee;
} t_date;


typedef struct
{
    char prenom[20];
    char nom[20];
    DATE dNaiss;
} t_etudiant;
```

```c
int main()
{
    t_etudiant * class;
    int i;
    class=(t_etudiant *) malloc(2*sizeof(t_etudiant));
    for(i=0;i<2;i++)
    {
        printf("Info étudiant %d\n", i+1);
        scanf("%s", class[i].prenom);
        scanf("%s", class[i].nom);
        scanf("%d/%d/%d", &class[i].dNaiss.jour, &class[i].dNaiss.mois , &class[i].dNaiss.annee);
    }

    for(i=0;i<2;i++)
    {
        printf("Info étudiant %d\n", i+1);
        printf("Prenom : %s\n", class[i].prenom);
        printf("Nom : %s\n", class[i].nom);
        printf("Né-e le : %d/%d/%d\n", class[i].dNaiss.jour, class[i].dNaiss.mois ,
class[i].dNaiss.annee);
    }
    free(class);

    return 0;
}
```

06

Les structures en paramètres de fonctions

06

Les structures en paramètres de fonctions

Définition

Si un paramètre d'une fonction est de type structuré, voir si c'est un :

- Passage par copie d'une structure
- Passage par copie d'adresse d'une structure (via un pointeur)

➔ Accéder aux attributs en fonction du type du paramètre.

Toujours se demander si la fonction a besoin de modifier ou non une variable pour faire le choix entre passer une copie de la variable ou bien une copie de son adresse

Les structures en paramètres de fonctions

Exemple:

```c
#include <stdio.h>

typedef struct
{
    int jour;
    int mois;
    int annee;
} t_date;


// Modification → on passe une copie d'adresse
void lire_date(t_date *d)
{
    printf("Saisir le jour : ");
    scanf("%d",&d->jour);

    printf("Saisir le mois : ");
    scanf("%d",&d->mois);

    printf("Saisir l'année : ");
    scanf("%d",&d->annee);
}
```

```c
// Pas de modification → on passe une copie
void afficher_date(t_date d)
{
    printf("%d/%d/%d \n", d.jour, d.mois, d.annee);
}

int main()
{
    t_date my_date;
    lire_date(&my_date);
    afficher_date(my_date);
    return 0;
}
```